归位 GuiWei · Bug 修复日志

真机测试反馈 · 自查与修复 · 验证记录

记录本轮全部缺陷的现象、根因、修复方案与验证结果

覆盖真机反馈 2 项 + 演示版/实用版分离 + 自查发现 6 项

环境：Xcode 27.0 · Swift 6.4 · iOS 27 SDK · iPhone 18 Pro 真机 + iPhone 18 Pro 模拟器

目 录

归位 GuiWei · Bug 修复日志 4

0.1 这一轮修了什么 4

0.2 修复原则 4

第 1 章 B1：标题与搜索栏重叠 5

1.1 现象 5

1.2 根因 5

1.3 修复 6

1.4 验证 6

第 2 章 B2：借出表单首次打开是空白 8

2.1 现象 8

2.2 根因 8

2.3 修复 8

2.4 验证 9

第 3 章 F1：演示版与实用版分离 11

3.1 需求 11

3.2 设计 11

3.3 分类为什么保留 11

3.4 防止"删光又长回来" 12

3.5 验证 12

3.6 F2：首次启动先询问外观偏好 13

3.7 F3：按时间段自动切换，以及怎么做到"不冲突" 13

第 4 章 自查发现的其他问题 18

4.1 B3：关掉提醒开关，通知照样会响 18

4.2 B4：删除后仍把旧数据写进 Spotlight 与小组件 18

4.3 B5：缩略图每帧重复解码 19

4.4 B6：数据库偶发打不开就删库 19

4.5 B7：实用版空状态没有指路 19

4.6 B8：位置递归统计缺少深度保护 19

第 5 章 验证记录 21

5.1 编译 21

5.2 验证清单 21

5.3 一个排查过程中的教训 21

第 6 章 遗留与建议 22

6.1 本轮没有做的事 22

6.2 建议的下一步 22

归位 GuiWei · Bug 修复日志

0.1 这一轮修了什么

这份日志记录 2026 年 10 月 9 日这一轮的缺陷处理。来源有三类：真机测试反馈、自查发现，以及新增的演示版需求。

编号	严重度	现象	状态
B1	高	主界面「归位」标题有时与搜索栏重叠卡住	已修复
B2	高	右滑物品选「借出」，前几次打开表单是空白	已修复
B3	中	关掉「临期与借出提醒」，已排程的通知照样会响	已修复
B4	中	删除物品后，Spotlight 索引与小组件快照仍写入旧数据	已修复
B5	中	列表缩略图每帧重复解码，真机滚动掉帧	已修复
B6	中	数据库偶发打不开就立刻删库，过于激进	已修复
B7	低	实用版空状态没有引导先去建位置	已修复
B8	低	位置数量递归统计缺少深度保护	已修复
F1	需求	演示版需自带数据，实用版需空白起步	已实现
F2	需求	打开 App 前先询问深色 / 浅色，默认跟随系统	已实现
F3	需求	增加「按时间段自动切换」，且不与已有选项冲突	已实现

0.2 修复原则

三条：**先复现、再定位根因、最后验证。**

凡是根因属于"两个状态之间有时间差"这类竞态，就改结构、让它在类型上不可能出错，而不是调参数去赌概率。下面 B2 就是典型。

第 1 章 B1：标题与搜索栏重叠

1.1 现象

真机（iPhone 18 Pro）测试时，主界面会出现「归位」大标题与搜索框叠在一起的情况：标题压在搜索框上层，两者占据同一块位置，有时会一直卡在这个状态不恢复。



真机上出现的重叠：标题压在搜索框上

1.2 根因

主界面原来的结构是这样的：

```
NavigationStack {
    VStack(spacing: 0) {
        filterBar
        content
    }
    .navigationTitle("归位")
    .searchable(text: searchBinding, prompt: "搜物品、位置或标签")
}
```

问题出在**把滚动视图包进了 `VStack`，同时又用了系统的「大标题 + `.searchable`」组合**。

`.searchable` 的搜索框属于导航栏，系统需要跟**直接的滚动视图**协调三件事：大标题的收缩、搜索框的显隐、内容的上边距。中间插一层 `VStack` 之后，这个协调关系被打断了——大标题的收缩动画和搜索框的布局各自为政，于是出现重叠，并且偶尔停在中间状态不再恢复。

所以这不是某个参数写错了，而是这个组合本身的薄弱点。

1.3 修复

思路是**彻底绕开这个状态**：不再使用系统的「大标题 + `.searchable`」，改成自绘头部。

```
NavigationStack(path: $path) {
    VStack(spacing: 0) {
        header      // 标题 + 三个操作按钮 + 搜索框 + 筛选条
        content
    }
    .toolbar(.hidden, for: .navigationBar)
}

private var header: some View {
    VStack(spacing: Theme.gapM) {
        HStack {
            Text("归位").font(.largeTitle).fontWeight(.bold)
            Spacer()
            Button { showingQuickFind = true } label: { /* 找东西 */ }
            Menu { Picker("排序", selection: $sortMode) { /* ... */ } } label: { }
            Button { showingEditor = true } label: { /* 添加 */ }
        }
        searchField     // 自绘搜索框
        filterBar       // 状态筛选条
    }
}
```

改完之后，标题、搜索框、筛选条**都只是普通视图**，由我们自己的 `VStack` 排版，不存在任何动画中间态，也就不可能重叠。

顺带把原来放在工具栏的三个按钮（找东西、排序、添加）移进了头部，位置和手感不变。

1.4 验证

修改后在模拟器上反复启动、滚动、切换标签，头部始终保持稳定状态。



修复后的主界面：标题、搜索框、筛选条各就各位

第 2 章 B2：借出表单首次打开是空白

2.1 现象

在物品列表**右滑**（从左往右划）露出「借出」按钮，点它打开的表单**前几次是空白的**——没有标题、没有输入框，就是一张空表单。多试几次之后又正常了。

2.2 根因

原来的代码用了两个独立的状态变量：

```
@State private var lendingItem: Item?
@State private var showingLendSheet = false

private func lendAction(_ item: Item) {
    lendingItem = item
    showingLendSheet = true
}

.sheet(isPresented: $showingLendSheet) {
    if let lendingItem {
        LendSheet(item: lendingItem)
    }
}
```

`showingLendSheet` 负责"要不要弹"，`lendingItem` 负责"弹出来给谁看"。**这是两个变量，SwiftUI 不保证它们在同一次更新里生效。**

滑动操作是在手势回调里触发的，两次赋值有可能落进不同的更新事务：`showingLendSheet` 先变成 `true`，系统立刻构建 sheet 内容，而此时 `lendingItem` 还是 `nil`，`if let` 不成立，于是弹出一张**没有内容的白纸**。

"前几次才出现"正好符合竞态的特征——它取决于那两次状态更新是否被合批处理。

2.3 修复

改用 `sheet(item:)`，把"要不要弹"和"给谁看"合并成**一个**状态：

```
@State private var lendingItem: Item?

private func lendAction(_ item: Item) {
    lendingItem = item
}

.sheet(item: $lendingItem) { item in
    LendSheet(item: item)
        .appSheetRadius()
}
```

`sheet(item:)` 的语义是：只要这个可选值非空，就带着**它本身**呈现表单。内容闭包拿到的 `item` 是解包后的非可选值，从类型上就不可能出现"弹了但没有对象"。

工程里位置页早就是这个写法（`.sheet(item: $editingSpot)`），这次把物品页统一过来。

2.4 验证

给物品列表加了一个调试入口，走**和真实滑动完全相同**的呈现路径：

```
xcrun simctl launch booted com.tianhaoming.guiwei -openLendSheet 1
```

修复后第一次打开就是完整的：标题「借出「滤芯备用装」」、借给谁、约定归还日期、应还日期 2026 年 10 月 16 日、到期前提醒我、备注，全部就位。



修复后的借出表单：第一次打开内容就是全的

第 3 章 F1：演示版与实用版分离

3.1 需求

- **演示版**：打开就自带一整套物品和位置，用于展示、汇报。
- **实用版**：位置与物品全部空白，等待使用者自行设定。

3.2 设计

新增 `AppDataMode`，两种形态：

形态	初始数据	用途
`demo`	预置 12 件物品、11 个位置、2 条借出记录	展示与汇报
`practical`	物品与位置为空	日常使用

默认值由编译条件 `DEMO_DATA` 决定：

```
static var current: AppDataMode {
    if let raw = UserDefaults.standard.string(forKey: modeKey),
       let mode = AppDataMode(rawValue: raw) {
        return mode
    }
    #if DEMO_DATA
    return .demo
    #else
    return .practical
    #endif
}
```

含义是：不加编译条件构建出来的是**实用版**，加了 `DEMO_DATA` 构建出来的是**演示版**。

同时设置页里也能随时切换，选择写在 UserDefaults 里、重启后保持。这样一个人用同一个包就能覆盖两种场景，不必维护两份工程。

3.3 分类为什么保留

清空只针对**物品、位置、借出记录**。

分类（食品、药品、数码、证件、衣物、工具、个护、其他）是 App 自带的分类体系，不属于示例内容，两种形态都需要，所以始终保留这 8 个默认分类。

3.4 防止"删光又长回来"

演示数据只装载一次，用 `hasLoadedDemoData` 标记记在 UserDefaults 里。

否则会出现这种情况：使用者在演示版里把东西全部删掉，下次启动数据又自己冒出来——那是个更让人困扰的 bug。

3.5 验证

在 iPhone 18 Pro 模拟器上全新安装后，分别验证两种形态，直接读 SQLite 计数：

形态	物品	位置	分类	借出
演示版	12	11	8	2
实用版	0	0	8	0



实用版全新安装后的样子：位置与物品都是空的

3.6 F2：首次启动先询问外观偏好

需求

打开 App 之前先问一次深色还是浅色，**默认跟随系统**。

实现

新增 `AppAppearance`，三个取值：

取值	含义	传给系统的值
`system`	跟随系统（默认）	`nil`
`light`	始终浅色	`.light`
`dark`	始终深色	`.dark`

偏好存在 `@AppStorage` 里，通过根视图上的一个修饰符即时生效：

```
.preferredColorScheme(AppAppearance(rawValue: appearanceRaw)?.colorScheme)
```

`system` 时传 `nil`，也就是把决定权交回系统——这正是"跟随系统"该有的语义。

流程怎么接

外观选择和新手指南放在**同一个 `fullScreenCover`** 里，用状态切换而不是"关掉一个再开一个"。后者在 SwiftUI 里容易出现第二个弹不出来的时序问题。

```
首次启动 → 选择外观 → 5 页新手指南 → 主界面
```

从设置页点「重新查看使用教程」时，只播指南、不再问外观（`skipAppearance` 参数控制）。

设置页也加了「外观」分组，可以随时改。

3.7 F3：按时间段自动切换，以及怎么做到"不冲突"

需求

在询问外观时，增加一个"允许按时间段自动切换"的选项，并且**不要和已有的功能冲突**。

为什么做成选项，而不是独立开关

如果把它做成一个独立的开关，就会出现这种自相矛盾的状态：

> 外观选了「深色」，同时又开着「按时间自动切换」

到时候到底听谁的？无论怎么定都是 bug。所以把它做成**同一个枚举里的一个取值**：

取值	含义
`system`	跟随系统（默认）
`auto`	按时间段自动切换
`light`	始终浅色
`dark`	始终深色

四者互斥，冲突从结构上就不存在了。

核心不变式：存偏好，不存结果

这是"不与其他功能冲突"最关键的一条：

```
/// 存储：始终是使用者的选择，可能是 auto
@AppStorage("guiwei.appearance") private var appearanceRaw

/// 运行时：按当下时间解析出来的方案，绝不写回存储
@State private var resolvedScheme: ColorScheme?
```

如果把解析结果写回去，晚上一到就会把使用者的"自动"改写成"深色"，第二天早上再也切不回浅色——那就成了一个只在一天里正确的 bug。所以**解析结果只活在内存里**。

时间段的定义

白天 07:00 到 19:00 用浅色，其余时间用深色。**用固定时间，不引入定位权限**——按日出日落切换虽然更精细，但要多申请一个权限，对一个物品管理 App 来说不划算。

三个会让它"失灵"的边界，都处理了

边界情况	不处理的后果	处理方式
App 一直开着，跨过切换点	界面停在旧外观	按距离下一个切换点的秒数安排一次性任务
设备休眠期间跨过切换点	定时器不触发，回到前台还是昨晚的深色	监听 `scenePhase`，回到前台重新解析
使用者改了偏好	旧的定时任务还在跑	每次重新解析前先取消上一个任务

验证

模拟器当前时间 10:27（白天）：

模式	列表区域平均亮度	判定
自动（按时间）	248.6	浅色，符合预期
手动深色	34.1	深色



自动模式在白天解析为浅色

已知的边界（明说，不藏着）

桌面小组件与实时活动跟随的是系统外观，不受 App 内的设置影响。 原因是它们跑在独立进程里，要读到 App 的偏好需要 App Group，而 App Group 要求付费开发者账号。

这个不一致无法在免费账号下消除，所以选择**在设置页里直接说明**，而不是让使用者自己去发现：

> 桌面小组件与实时活动由系统单独绘制，始终跟随系统外观，不受这里影响。

验证（外观选择的默认值）

场景	结果
全新安装后启动	先出现「选择外观」，默认选中「跟随系统」
强制深色	界面整体转为深色
强制浅色	界面整体转为浅色



全新安装后先问外观



同一次构建下，深色与浅色的实际效果

第 4 章 自查发现的其他问题

4.1 B3：关掉提醒开关，通知照样会响

现象：在设置页把「临期与借出提醒」关掉，界面只弹了一句"提醒已关闭"，但之前排好的通知到点仍然会响。

根因：关闭分支里只有 `showToast`，没有撤销任何已排程的通知。

```
} else {
    showToast("提醒已关闭")      // 只说了句话，什么都没做
}
```

而且这个开关当时是直接绑在**系统授权状态**上的，它本身就表达不了"我不想收提醒"——App 没有能力把系统授权改回去，所以拨到关闭其实是个假动作。

修复：把"使用者的偏好"和"系统权限"拆成两件事。

- 新增 `remindersEnabled`（`@AppStorage` 持久化），开关绑的是它；
- 关闭时调用 `cancelAll()`，真正撤销所有已排程与已送达的通知；
- 启动时的 `rescheduleExpiry` 先检查这个偏好，关掉之后不会再偷偷排回来；
- 系统权限只负责显示状态，以及授权被拒时给一个"去系统设置里打开"的入口。

4.2 B4：删除后仍把旧数据写进 Spotlight 与小组件

现象：删掉一件物品后，Spotlight 里还能搜到它，桌面小组件的数字也没变化。

根因：删除后的同步用了 `@Query` 的属性。

```
private func refreshEcosystem() {
    SpotlightIndexer.reindex(items: items)   // items 还是删除前的快照
    WidgetBridge.publish(items: items)
}
```

`context.delete` 加 `save` 之后，`@Query` 的刷新要等下一个更新周期，而这两行是立刻执行的，于是把**已删除的物品又写了一遍**。

修复：改为重新从上下文取一次。

```
let current = (try? context.fetch(FetchDescriptor<Item>())) ?? items
SpotlightIndexer.reindex(items: current)
WidgetBridge.publish(items: current)
```

4.3 B5：缩略图每帧重复解码

现象：真机上物品列表滚动不够跟手，图片越多越明显。

根因：列表行的缩略图在 `body` 里直接解码。

```
if let data = item.thumbnailData, let image = UIImage(data: data) {
```

存的确实已经是 300 像素的小图（这一点原本就做对了），但 SwiftUI 滚动时会反复求值 `body`，每次都要重新做一遍 JPEG 解码。这是纯粹的重复劳动。

修复：加一层解码缓存。用 `uid` 加数据长度做键——图片被替换时长度几乎一定变化，缓存自然失效；`NSCache` 本身线程安全，内存吃紧时也会自动回收。

4.4 B6：数据库偶发打不开就删库

现象：这是上一轮为应对模型变更加的兜底逻辑，但它在**第一次**打开失败时就会删掉整个本地库。

风险：首次打开失败有可能只是一次瞬时 I/O 问题。为了一次偶发失败就清空使用者的全部数据，代价太大——而"删库"是用户完全无法自救的操作。

修复：改成**连续两次打不开才删库重建**，最后仍然保留内存库兜底，保证绝不会启动失败。

4.5 B7：实用版空状态没有指路

现象：实用版第一次打开，界面提示"还没有记录任何物品"，但此时**一个位置都没有**。点「添加第一件物品」进去，会发现选不到存放位置。

修复：空状态文案按"有没有建过位置"分流。没有位置时提示：

> 先到「位置」页建一个房间和容器，再回来记录东西放在哪。

4.6 B8：位置递归统计缺少深度保护

现象：位置路径 `fullPath` 有 8 层深度保护，但数量统计是裸递归。

```
var totalItemCount: Int {
    items.count + children.reduce(0) { $0 + $1.totalItemCount }
}
```

位置编辑页已经排除了"把自己或自己的后代设为上级"，所以正常操作不会成环。但递归统计本身没有任何保护，一旦数据被外部导入成环就是死循环。

修复：加深度上限，超限即停止递归。属于防御性修复。

第 5 章 验证记录

5.1 编译

改完后完整构建：

```
xcodebuild build -project GuiWei.xcodeproj -scheme GuiWei \
  -configuration Debug -sdk iphonesimulator \
  -destination 'generic/platform=iOS Simulator' \
  -derivedDataPath "$PWD/.build" \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO \
  OTHER_SWIFT_FLAGS="-disable-sandbox"
```

结果：`** BUILD SUCCEEDED **`，零错误零警告。

5.2 验证清单

验证项	方法	结果
头部不再重叠	反复启动、滚动、切换标签	稳定，未复现
借出表单不空白	走与滑动相同的呈现路径	首次即完整
演示版数据	读 SQLite 计数	12 / 11 / 8 / 2
实用版空白	全新安装后读计数	0 / 0 / 8 / 0
空状态引导	全新安装后看首页	显示建位置引导

5.3 一个排查过程中的教训

这轮中途出现过一次"App 装好了但点不开"，报 `Simulator device failed to launch`，排查了许久才反应过来：

Bundle ID 已经为真机签名改成了 `com.tianhaoming.guiwei`，而我的验证脚本还在用旧的 `com.bingyan.guiwei` 去启动。根本不是崩溃，是去启动一个不存在的 App。

记在这里提醒自己：验证脚本里的标识要从工程里读，不要写死。

第 6 章 遗留与建议

6.1 本轮没有做的事

独立的两个 App。目前是"一个包、两种形态、设置页可切"。如果确实需要两个能并排安装的 App（各自的 Bundle ID 与应用名），需要再加一套构建配置。这件事没有做，原因是它会多出两个 Bundle ID，免费开发者账号的签名流程会更容易出错——对现在的使用场景来说，设置页一键切换更省事。

UI 自动化测试。像借出表单这类竞态缺陷，最适合用 UI 测试锁死，但项目还没有测试 target。

6.2 建议的下一步

1. 在真机上把两种形态各跑一遍，重点观察 B1 在长时间使用后是否真的不再出现。
2. 演示前提前在演示版里把数据过一遍，确认没有展示不出来的字段。
3. 免费证书 7 天到期，演示前一天重新运行一次，避免现场打不开。